SZABIST
University

Shaheed Zulfikar Ali Bhutto
Institute of Science and Technology

ROBOTICS & ARTIFICIAL INTELLIGENCE DEPARTMENT

Total Marks:_____

Obtained Marks: _____

Computing Vision

Lab Task # 11

Submitted To: **Mr. Muhammad Azhar**

Student Name: **Habiba Bibi**

Reg. Number: **23108111**

Code Example: Object Detection in Visual Scenes Using Particle Swarm Optimization (PSO)

In many real-world applications like surveillance, robot vision, or industrial inspection, object detection must be performed on scenes where the object might appear in different locations, potentially with noise, shadows, or background clutter. Traditional template matching can be computationally expensive when exhaustively searching all positions. To overcome this challenge, Particle Swarm Optimization (PSO) is applied to reduce the search space and efficiently locate the target object (template) in a given scene.

```
# Code Example: Object Detection in Visual Scenes Using Particle Swarm Optimization (PSO)
import cv2
import numpy as np
import random
import matplotlib.pyplot as plt
img = cv2.imread("D:/23108111/H18.jpg", 0)
template = cv2.imread("D:/23108111/HH4.jpg", 0)
img = cv2.resize(img, None, fx=0.5, fy=0.5)
th, tw = template.shape
ih, iw = img.shape
if tw > iw or th > ih:
    template = cv2.resize(template, (iw // 2, ih // 2))
    th, tw = template.shape
def fitness(x, y):
    roi = img[y:y+th, x:x+tw]
    return cv2.matchTemplate(roi, template, cv2.TM_CCORR_NORMED)[0][0]
num_particles = 5
iterations = 10
w = 0.5
c1 = 1
c2 = 2
positions = [
    np.array([random.randint(0, iw - tw),random.randint(0, ih - th)])for _ in range(num_particles)]
velocities = [np.zeros(2) for _ in range(num_particles)]
pBests = positions.copy()
pBest_scores = [fitness(*p) for p in pBests]
gBest = pBests[np.argmax(pBest_scores)]
gBest_score = max(pBest_scores)
for _ in range(iterations):
    for i in range(num_particles):
        r1 = random.random()
        r2 = random.random()
        velocities[i] = (w * velocities[i]+ c1 * r1 * (pBests[i] - positions[i])+ c2 * r2 * (gBest - positions[i]))
        positions[i] = np.clip(
            positions[i] + velocities[i],[0, 0],[iw - tw, ih - th]).astype(int)
        score = fitness(*positions[i])
        if score > pBest_scores[i]:
            pBests[i] = positions[i]
```

```
        if score > pBest_scores[i]:
            pBests[i] = positions[i]
            pBest_scores[i] = score
        if score > gBest_score:
            gBest = positions[i]
            gBest_score = score
result = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
cv2.rectangle(result, tuple(gBest), (gBest[0] + tw, gBest[1] + th), (0, 255, 0), 2)
print("Best Match Score:", gBest_score)
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.show()
```

```
Best Match Score: 0.85436827
```

Code Example: Object Localization Using PSO on Ackley Function

In robotics and autonomous systems, identifying optimal points in a 2D or 3D environment for object placement, navigation, or localization is a common task. When the environment has multiple local minima and a complex surface, traditional optimization algorithms often fail to find the global best solution.

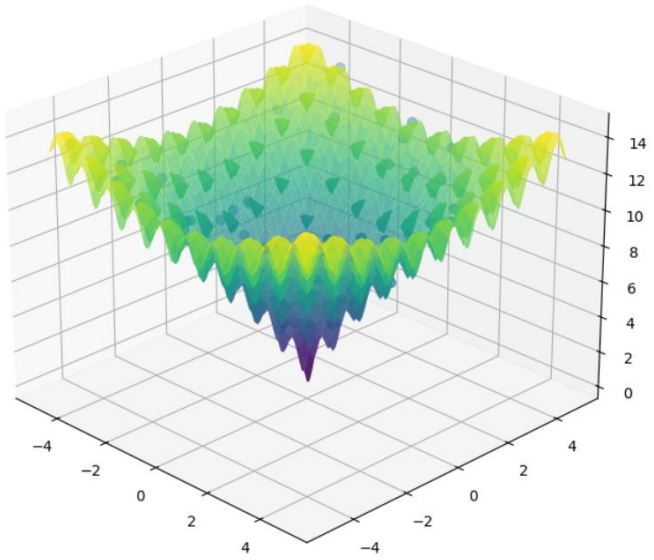
To address this, Particle Swarm Optimization (PSO), a nature-inspired global optimization algorithm, is used for object detection and localization tasks in synthetic terrains.

In this case study, a 2D synthetic surface (Ackley function) is used to simulate a challenging optimization landscape. This environment mimics real-world terrains (e.g., valleys, peaks, flatlands), and the task is to detect the global minimum (interpreted as the most optimal object location) using PSO.

```
# Code Example: Object Localization Using PSO on Ackley Function
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import animation
from mpl_toolkits.mplot3d import Axes3D
def ackley_fun(x):
    return (-20 * np.exp(-0.2 * np.sqrt(0.5 * (x[0]**2 + x[1]**2)))- np.exp(0.5 * (np.cos(2 * np.pi * x[0]) + np.cos(2 * np.pi * x[1])))+ np.exp(1)+ 20)
def pso2D(fitness_fun,data_range,num_particles=100,num_iterations=50,c1=0.8,c2=0.9,w=0.5):
    population = np.random.uniform(low=data_range[0],high=data_range[1],size=(num_particles, 2))
    velocities = np.zeros((num_particles, 2))
    personal_bests = np.full(num_particles, np.inf)
    personal_best_solutions = np.copy(population)
    global_best = np.inf
    best_solution = population[0]
    history = []
    for iteration in range(num_iterations):
        history.append(np.copy(population))
        for j in range(num_particles):
            score = fitness_fun(population[j])
            if score < personal_bests[j]:
                personal_bests[j] = score
                personal_best_solutions[j] = population[j]
        if np.min(personal_bests) < global_best:
            global_best = np.min(personal_bests)
            best_solution = personal_best_solutions[np.argmin(personal_bests)]
        print(f"Iteration {iteration+1}, "f"Best Fitness: {global_best:.5f}, "f"Best Solution: {best_solution}")
        for i, particle in enumerate(population):
            velocities[i] = (w * velocities[i]+ c1 * np.random.rand() * (personal_best_solutions[i] - particle)+ c2 * np.random.rand() * (best_solution
            population[i] = particle + velocities[i]
    return history
history = pso2D(ackley_fun,(-5, 5),num_particles=50,num_iterations=30)
x = np.linspace(-5, 5, 100)
y = np.linspace(-5, 5, 100)
X, Y = np.meshgrid(x, y)
Z = np.array([[ackley_fun([x, y]) for x in x]for y in y])
fig = plt.figure(figsize=(12, 8))
ax = fig.add_subplot(111, projection='3d')
ax.plot_surface(X,Y,Z,cmap='viridis',edgecolor='none',alpha=0.7)
```

```
ax.plot_surface(X,Y,Z,cmap='viridis',edgecolor='none',alpha=0.7)
data = history[0]
scat = ax.scatter(data[:, 0],data[:, 1],[ackley_fun(p) for p in data],s=40)
def animate(frame):
    data = history[frame]
    scat._offsets3d = (data[:, 0],data[:, 1],[ackley_fun(p) for p in data])
    ax.set_title(f"Iteration {frame+1}")
    return scat,
ax.view_init(25, -45)
ani = animation.FuncAnimation(fig,animate,frames=len(history),interval=300,blit=False)
plt.show()
```

```
Iteration 1, Best Fitness: 4.14442, Best Solution: [ 0.1319767  -1.18943678]
Iteration 2, Best Fitness: 1.17491, Best Solution: [-0.01126046  0.17886379]
Iteration 3, Best Fitness: 1.17491, Best Solution: [-0.01126046  0.17886379]
Iteration 4, Best Fitness: 0.20700, Best Solution: [-0.02361821 -0.04410689]
Iteration 5, Best Fitness: 0.06805, Best Solution: [-0.01170876  0.01648447]
Iteration 6, Best Fitness: 0.02661, Best Solution: [-0.00787286 -0.00369485]
Iteration 7, Best Fitness: 0.02661, Best Solution: [-0.00787286 -0.00369485]
Iteration 8, Best Fitness: 0.02661, Best Solution: [-0.00787286 -0.00369485]
Iteration 9, Best Fitness: 0.02661, Best Solution: [-0.00787286 -0.00369485]
Iteration 10, Best Fitness: 0.00275, Best Solution: [0.00094713 0.00016879]
Iteration 11, Best Fitness: 0.00275, Best Solution: [0.00094713 0.00016879]
Iteration 12, Best Fitness: 0.00275, Best Solution: [0.00094713 0.00016879]
Iteration 13, Best Fitness: 0.00275, Best Solution: [0.00094713 0.00016879]
Iteration 14, Best Fitness: 0.00275, Best Solution: [0.00094713 0.00016879]
Iteration 15, Best Fitness: 0.00275, Best Solution: [0.00094713 0.00016879]
Iteration 16, Best Fitness: 0.00160, Best Solution: [-1.20408807e-05  5.62662648e-04]
Iteration 17, Best Fitness: 0.00146, Best Solution: [0.00010513 0.00050278]
Iteration 18, Best Fitness: 0.00146, Best Solution: [0.00010513 0.00050278]
Iteration 19, Best Fitness: 0.00146, Best Solution: [0.00010513 0.00050278]
Iteration 20, Best Fitness: 0.00131, Best Solution: [-0.00028107  0.00036542]
Iteration 21, Best Fitness: 0.00025, Best Solution: [ 6.96824880e-05 -5.17669616e-05]
Iteration 22, Best Fitness: 0.00024, Best Solution: [-1.05993794e-05  8.49039027e-05]
Iteration 23, Best Fitness: 0.00014, Best Solution: [1.94082464e-05 4.41114903e-05]
Iteration 24, Best Fitness: 0.00014, Best Solution: [1.94082464e-05 4.41114903e-05]
Iteration 25, Best Fitness: 0.00013, Best Solution: [-2.9435142e-05  3.5549949e-05]
Iteration 26, Best Fitness: 0.00005, Best Solution: [ 4.83621120e-07 -1.74658912e-05]
Iteration 27, Best Fitness: 0.00003, Best Solution: [ 1.04949601e-05 -3.56564440e-06]
```



Optimizing Bounding Box Localization in Satellite Imagery Using Particle Swarm Optimization

Problem Context: In low-resolution satellite imagery, detecting small man-made structures like buildings, vehicles, or tents is challenging due to varying scales, lighting, and surrounding textures. Traditional sliding-window techniques are computationally expensive and may fail to adapt to diverse terrains.

Proposed Solution: Using Particle Swarm Optimization (PSO), candidate bounding box positions are treated as particles. Each particle explores the image space, searching for areas with high texture similarity or edge density compared to a known template of the object (e.g., a vehicle roof). PSO helps quickly converge to the region with the highest fitness, identifying potential targets efficiently.

Implementation Strategy:

□ Satellite image (scene_satellite.jpg) and a sample object template (template_vehicle.jpg) are inputs.

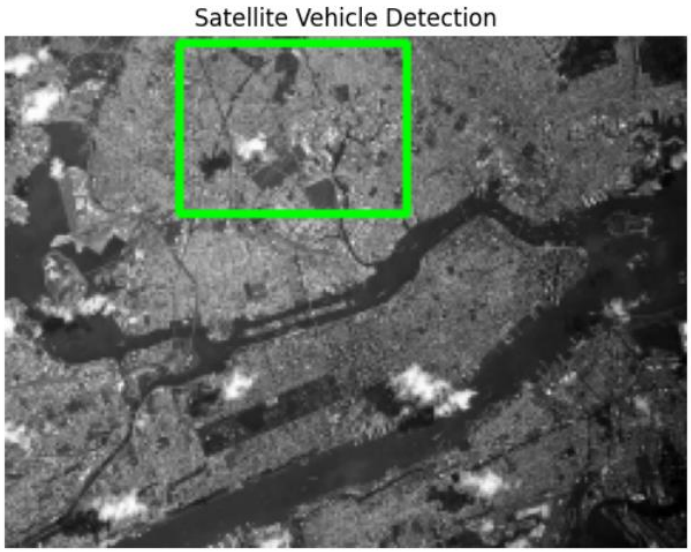
□ The fitness function measures edge density or template correlation.

□ PSO updates particle positions to find the most likely match region.

Real-World Application: Used in disaster response, military reconnaissance, or urban planning, where automated object detection on satellite images is essential for fast decision-making.

```python
# Optimizing Bounding Box Localization in Satellite Imagery Using Particle Swarm Optimization
import cv2
import numpy as np
import random
import matplotlib.pyplot as plt
img = cv2.imread("D:/23108111/H7.jpg", 0)
template = cv2.imread("D:/23108111/HH4.jpg", 0)
img = cv2.resize(img, None, fx=0.5, fy=0.5)
ih, iw = img.shape
th, tw = template.shape
if tw > iw or th > ih:
    template = cv2.resize(template, (iw // 3, ih // 3))
th, tw = template.shape
def fitness(x, y):
    roi = img[y:y+th, x:x+tw]
    edges = cv2.Canny(roi, 100, 200)
    temp_edges = cv2.Canny(template, 100, 200)
    return cv2.matchTemplate(edges,temp_edges,cv2.TM_CCORR_NORMED)[0][0]
num_particles = 10
iterations = 20
w = 0.5
c1 = 1
c2 = 2
positions = [
    np.array([random.randint(0, iw - tw),random.randint(0, ih - th) ])for _ in range(num_particles)]
velocities = [np.zeros(2) for _ in range(num_particles)]
pBests = positions.copy()
pBest_scores = [fitness(*p) for p in pBests]
gBest = pBests[np.argmax(pBest_scores)]
gBest_score = max(pBest_scores)
for _ in range(iterations):
    for i in range(num_particles):
        r1 = random.random()
        r2 = random.random()
        velocities[i] = (w * velocities[i]+ c1 * r1 * (pBests[i] - positions[i])+ c2 * r2 * (gBest - positions[i]))
        positions[i] = np.clip(positions[i] + velocities[i],[0, 0],[iw - tw, ih - th]).astype(int)
        score = fitness(*positions[i])
        if score > pBest_scores[i]:
            pBests[i] = positions[i]
            pBest_scores[i] = score
        if score > gBest_score:
            gBest = positions[i]
            gBest_score = score
result = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
cv2.rectangle(result,tuple(gBest),(gBest[0] + tw, gBest[1] + th),(0, 255, 0),2)
print("Best Match Score:", gBest_score)
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title("Satellite Vehicle Detection")
plt.axis("off")
plt.show()
```

```
Best Match Score: 0.29960376
```



Tumor Pattern Detection in MRI Using Particle Swarm Optimization-Based Template Matching

Problem Context: In radiology, identifying the location of a known anomaly (e.g., tumor, cyst, or lesion) in an MRI scan requires expertise and time. Due to anatomical variability between patients, such patterns may appear in different regions and sizes across individuals.

Proposed Solution: PSO is used to automate the search for known pathological structures in grayscale MRI slices. A template of the anomaly (e.g., a tumor from a prior scan) is matched to the new scan, and PSO optimizes its position by maximizing template similarity.

Implementation Strategy:

□ MRI scan (brain_scan.jpg) and tumor template (tumor_template.jpg) are input images.

□ The fitness function could use normalized cross-correlation to measure similarity.

□ The PSO algorithm dynamically navigates the image to localize the most probable tumor region.

Real-World Application: Assists radiologists and AI-aided diagnostic systems in improving accuracy and reducing time in identifying critical regions in medical images.

```
# Tumor Pattern Detection in MRI Using Particle Swarm Optimization-Based Template Matching
import cv2
import numpy as np
import random
import matplotlib.pyplot as plt
img_path=r"D:/23108111/H20.jpeg"
temp_path=r"D:/23108111/H35.jpg"
img=cv2.imread(img_path,0)
template=cv2.imread(temp_path,0)
if img is None:
    raise Exception("Main image not found")
if template is None:
    raise Exception("Template image not found")
img=cv2.resize(img,(0,0),fx=0.5,fy=0.5)
template=cv2.resize(template,(0,0),fx=0.5,fy=0.5)
h,w=img.shape
th,tw=template.shape
if th>=h or tw>=w:
    scale=min((h-1)/th,(w-1)/tw)
    img=cv2.resize(img,(0,0),fx=scale,fy=scale)
    template=cv2.resize(template,(0,0),fx=scale,fy=scale)
    h,w=img.shape
    th,tw=template.shape
def fitness(x,y):
    x=int(x)
    y=int(y)
    if x<0 or y<0 or x+tw>w or y+th>h:
        return float('inf')
    roi=img[y:y+th,x:x+tw]
    if roi.shape!=template.shape:
        return float('inf')
    return np.sum((roi-template)**2)
num_particles=10
iterations=20
w_inertia=0.5
c1=1
c2=2
positions=[]
```

```
gBest_score=min(pBest_scores)
for _ in range(iterations):
    for i in range(num_particles):
        r1=random.random()
        r2=random.random()
        velocities[i]=(w_inertia*velocities[i]+c1*r1*(pBests[i]-positions[i])+c2*r2*(gBest-positions[i]))
        positions[i]=positions[i]+velocities[i]
        positions[i][0]=np.clip(positions[i][0],0,w-tw)
        positions[i][1]=np.clip(positions[i][1],0,h-th)
        score=fitness(positions[i][0],positions[i][1])
        if score<pBest_scores[i]:
            pBests[i]=positions[i].copy()
            pBest_scores[i]=score
        if score<gBest_score:
            gBest=positions[i].copy()
            gBest_score=score
result=cv2.cvtColor(img,cv2.COLOR_GRAY2BGR)
x,y=int(gBest[0]),int(gBest[1])
cv2.rectangle(result,(x,y),(x+tw,y+th),(0,255,0),2)
print("Best Match Score:",gBest_score)
plt.imshow(cv2.cvtColor(result,cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.title("PSO Detection (Safe Version)")
plt.show()
```

```
Best Match Score: inf
```

Case Study 1: Tumor Localization in MRI Scans
Title: PSO-Based Optimization for Tumor Region Detection in Brain MRI Images

Scenario: In medical imaging, accurately detecting tumors in MRI scans is critical. However, tumors vary in size and position across patients.
Implementation:
□ A known tumor pattern (template) is used.
□ PSO searches the MRI scan space to find the best matching region using template matching with correlation-based fitness.
□ Helps automate and accelerate diagnosis.

Real-world application: Clinical decision support systems in radiology for brain tumor detection.

```
# Case Study 1: Tumor Localization in MRI Scans
# Tumor Pattern Detection in MRI Using Particle Swarm Optimization-Based Template Matching
import cv2
import numpy as np
import random
import matplotlib.pyplot as plt
img_path=r"D:/23108111/H20.jpeg"
temp_path=r"D:/23108111/H35.jpg"
img=cv2.imread(img_path,0)
template=cv2.imread(temp_path,0)
if img is None:
    raise Exception("Main image not found")
if template is None:
    raise Exception("Template image not found")
img=cv2.resize(img,(0,0),fx=0.5,fy=0.5)
template=cv2.resize(template,(0,0),fx=0.5,fy=0.5)
h,w=img.shape
th,tw=template.shape
if th>=h or tw>=w:
    scale=min((h-1)/th,(w-1)/tw)
    img=cv2.resize(img,(0,0),fx=scale,fy=scale)
    template=cv2.resize(template,(0,0),fx=scale,fy=scale)
    h,w=img.shape
    th,tw=template.shape
def fitness(x,y):
    x=int(x)
    y=int(y)
    if x<0 or y<0 or x+tw>w or y+th>h:
        return float('inf')
    roi=img[y:y+th,x:x+tw]
    if roi.shape!=template.shape:
        return float('inf')
    return np.sum((roi-template)**2)
num_particles=10
iterations=20
w_inertia=0.5
c1=1
c2=2
```

```
        velocities[i]=(w_inertia*velocities[i]+c1*r1*(pBests[i]-positions[i])+c2*r2*(gBest-positions[i]))
        positions[i]=positions[i]+velocities[i]
        positions[i][0]=np.clip(positions[i][0],0,w-tw)
        positions[i][1]=np.clip(positions[i][1],0,h-th)
        score=fitness(positions[i][0],positions[i][1])
        if score<pBest_scores[i]:
            pBests[i]=positions[i].copy()
            pBest_scores[i]=score
        if score<gBest_score:
            gBest=positions[i].copy()
            gBest_score=score
result=cv2.cvtColor(img,cv2.COLOR_GRAY2BGR)
x,y=int(gBest[0]),int(gBest[1])
cv2.rectangle(result,(x,y),(x+tw,y+th),(0,255,0),2)
print("Best Match Score:",gBest_score)
plt.imshow(cv2.cvtColor(result,cv2.COLOR_BGR2RGB))
plt.axis("off")
plt.title("PSO Detection (Safe Version)")
plt.show()
```

```
Best Match Score: inf
```

Case Study 2: Vehicle Detection in Aerial/Satellite Imagery
Title: PSO-Based Vehicle Detection in Satellite Surveillance Images
Scenario: Defense and traffic monitoring applications require detecting vehicles from aerial views.
Implementation:
□ A satellite image and a vehicle roof template are inputs.
□ PSO searches for areas that closely match the vehicle template using texture or shape descriptors.
□ Fitness function uses normalized cross-correlation or edge similarity.

Real-world application: Military reconnaissance, border surveillance, and disaster zone assessment.

```python
# Case Study 2: Vehicle Detection in Aerial/Satellite Imagery
import cv2
import numpy as np
import random
import matplotlib.pyplot as plt
img = cv2.imread("D:/23108111/H7.jpg", 0)
template = cv2.imread("D:/23108111/HH4.jpg", 0)
img = cv2.resize(img, None, fx=0.5, fy=0.5)
ih, iw = img.shape
th, tw = template.shape
if tw > iw or th > ih:
    template = cv2.resize(template, (iw // 3, ih // 3))
th, tw = template.shape
def fitness(x, y):
    roi = img[y:y+th, x:x+tw]
    edges = cv2.Canny(roi, 100, 200)
    temp_edges = cv2.Canny(template, 100, 200)
    return cv2.matchTemplate(edges,temp_edges,cv2.TM_CCORR_NORMED)[0][0]
num_particles = 10
iterations = 20
w = 0.5
c1 = 1
c2 = 2
positions = [
    np.array([random.randint(0, iw - tw),random.randint(0, ih - th) ])for _ in range(num_particles)]
velocities = [np.zeros(2) for _ in range(num_particles)]
pBests = positions.copy()
pBest_scores = [fitness(*p) for p in pBests]
gBest = pBests[np.argmax(pBest_scores)]
gBest_score = max(pBest_scores)
for _ in range(iterations):
    for i in range(num_particles):
        r1 = random.random()
        r2 = random.random()
        velocities[i] = (w * velocities[i]+ c1 * r1 * (pBests[i] - positions[i])+ c2 * r2 * (gBest - positions[i]))
        positions[i] = np.clip(positions[i] + velocities[i],[0, 0],[iw - tw, ih - th]).astype(int)
        score = fitness(*positions[i])
        if score > pBest_scores[i]:
            pBests[i] = positions[i]
            pBest_scores[i] = score
        if score > gBest_score:
            gBest = positions[i]
            gBest_score = score
result = cv2.cvtColor(img, cv2.COLOR_GRAY2BGR)
cv2.rectangle(result,tuple(gBest),(gBest[0] + tw, gBest[1] + th),(0, 255, 0),2)
print("Best Match Score:", gBest_score)
plt.imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
plt.title("Satellite Vehicle Detection")
plt.axis("off")
plt.show()
```

```
Best Match Score: 0.30849546
```


```python
# Case Study 3: Industrial Object Detection in Conveyor Belt Systems
```

Case Study 3: Industrial Object Detection in Conveyor Belt Systems
Title: PSO for Object Presence Verification in Real-Time Industrial Vision Systems
Scenario: In a production line, components must be verified for correct placement, shape, or orientation.
Implementation:
□ Each product image is scanned for the presence of a part using a known template.
□ PSO rapidly explores the image for a match, ensuring speed and accuracy.
□ Reduces reliance on exhaustive scanning.

Real-world application: Used in automated visual inspection systems for manufacturing and assembly line QA.

```
# Case Study 3: Industrial Object Detection in Conveyor Belt Systems
import cv2
import numpy as np
import random
import matplotlib.pyplot as plt
img = cv2.imread("D:/23108111/H37.jpg", 0)
template = cv2.imread("D:/23108111/H36.jpg", 0)
if img is None:
    raise Exception("Conveyor image not found")
if template is None:
    raise Exception("Template image not found")
h,w=img.shape
th,tw=template.shape
if th>=h or tw>=w:
    scale=min((h-1)/th,(w-1)/tw)
    template=cv2.resize(template,(0,0),fx=scale,fy=scale)
    th,tw=template.shape
def fitness(x,y):
    x=int(x)
    y=int(y)
    if x<0 or y<0 or x+tw>w or y+th>h:
        return -1
    roi=img[y:y+th,x:x+tw]
    if roi.shape!=template.shape:
        return -1
    score=cv2.matchTemplate(roi,template,cv2.TM_CCOEFF_NORMED)[0][0]
    return score
num_particles=15
iterations=25
w_inertia=0.5
c1=1
c2=2
positions=[]
for _ in range(num_particles):
    positions.append(
        np.array([random.randint(0,w-tw),random.randint(0,h-th)],dtype=float))
velocities=[np.zeros(2) for _ in range(num_particles)]
```

```
        score=fitness(positions[i][0],positions[i][1])
        if score>pBest_scores[i]:
            pBests[i]=positions[i].copy()
            pBest_scores[i]=score
        if score>gBest_score:
            gBest=positions[i].copy()
            gBest_score=score
result=cv2.cvtColor(img,cv2.COLOR_GRAY2BGR)
x,y=int(gBest[0]),int(gBest[1])
cv2.rectangle(result,(x,y),(x+tw,y+th),(0,255,0),2)
print("Best Match Score:",gBest_score)
if gBest_score>0.7:
    print("Component Detected")
else:
    print("Component Not Detected")
plt.imshow(cv2.cvtColor(result,cv2.COLOR_BGR2RGB))
plt.title("Industrial Object Detection using PSO")
plt.axis("off")
plt.show()
```

```
Best Match Score: 0.052756548
Component Not Detected
```

Case Study 4: Wildlife Recognition in Forest Surveillance Footage

Title: Locating Endangered Species in Forest Camera Trap Images Using PSO
Scenario: Camera traps in forests capture thousands of images. Locating specific animal species like tigers or elephants from these images is labor-intensive.
Implementation:
□ Use an animal face or body pattern as a template.
□ PSO optimizes the location where the template best fits using texture matching.
□ Enables semi-automated species detection.

Real-world application: Wildlife conservation monitoring and animal movement tracking.

```python
# Case Study 4: Wildlife Recognition in Forest Surveillance Footage
import cv2
import numpy as np
import random
import matplotlib.pyplot as plt
img = cv2.imread("D:/23108111/H38.jpg", 0)
template = cv2.imread("D:/23108111/H31.jpg", 0)
if img is None:
    raise Exception("Forest image not found")
if template is None:
    raise Exception("Animal template not found")
h,w=img.shape
th,tw=template.shape
if th>=h or tw>=w:
    scale=min((h-1)/th,(w-1)/tw)
    template=cv2.resize(template,(0,0),fx=scale,fy=scale)
    th,tw=template.shape
img_blur=cv2.GaussianBlur(img,(5,5),0)
template_blur=cv2.GaussianBlur(template,(5,5),0)
def fitness(x,y):
    x=int(x)
    y=int(y)
    if x<0 or y<0 or x+tw>w or y+th>h:
        return -1
    roi=img_blur[y:y+th,x:x+tw]
    if roi.shape!=template_blur.shape:
        return -1
    score=cv2.matchTemplate(roi,template_blur,cv2.TM_CCOEFF_NORMED)[0][0]
    return score
num_particles=15
iterations=25
w_inertia=0.5
c1=1
c2=2
positions=[]
for _ in range(num_particles):
    positions.append(
        np.array([random.randint(0,w-tw),random.randint(0,h-th)],dtype=float))
```

```python
        np.array([random.randint(0,w-tw),random.randint(0,h-th)],dtype=float))
velocities=[np.zeros(2) for _ in range(num_particles)]
pBests=positions.copy()
pBest_scores=[fitness(p[0],p[1]) for p in pBests]
gBest=pBests[np.argmax(pBest_scores)]
gBest_score=max(pBest_scores)
for _ in range(iterations):
    for i in range(num_particles):
        r1=random.random()
        r2=random.random()
        velocities[i]=(w_inertia*velocities[i]+c1*r1*(pBests[i]-positions[i])+c2*r2*(gBest-positions[i]))
        positions[i]=positions[i]+velocities[i]
        positions[i][0]=np.clip(positions[i][0],0,w-tw)
        positions[i][1]=np.clip(positions[i][1],0,h-th)
        score=fitness(positions[i][0],positions[i][1])
        if score>pBest_scores[i]:
            pBests[i]=positions[i].copy()
            pBest_scores[i]=score
        if score>gBest_score:
            gBest=positions[i].copy()
            gBest_score=score
result=cv2.cvtColor(img,cv2.COLOR_GRAY2BGR)
x,y=int(gBest[0]),int(gBest[1])
cv2.rectangle(result,(x,y),(x+tw,y+th),(0,255,0),2)
print("Best Match Score:",gBest_score)
if gBest_score>0.7:
    print("Animal Detected")
else:
    print("Animal Not Detected")
plt.imshow(cv2.cvtColor(result,cv2.COLOR_BGR2RGB))
plt.title("Wildlife Recognition using PSO")
plt.axis("off")
plt.show()
```

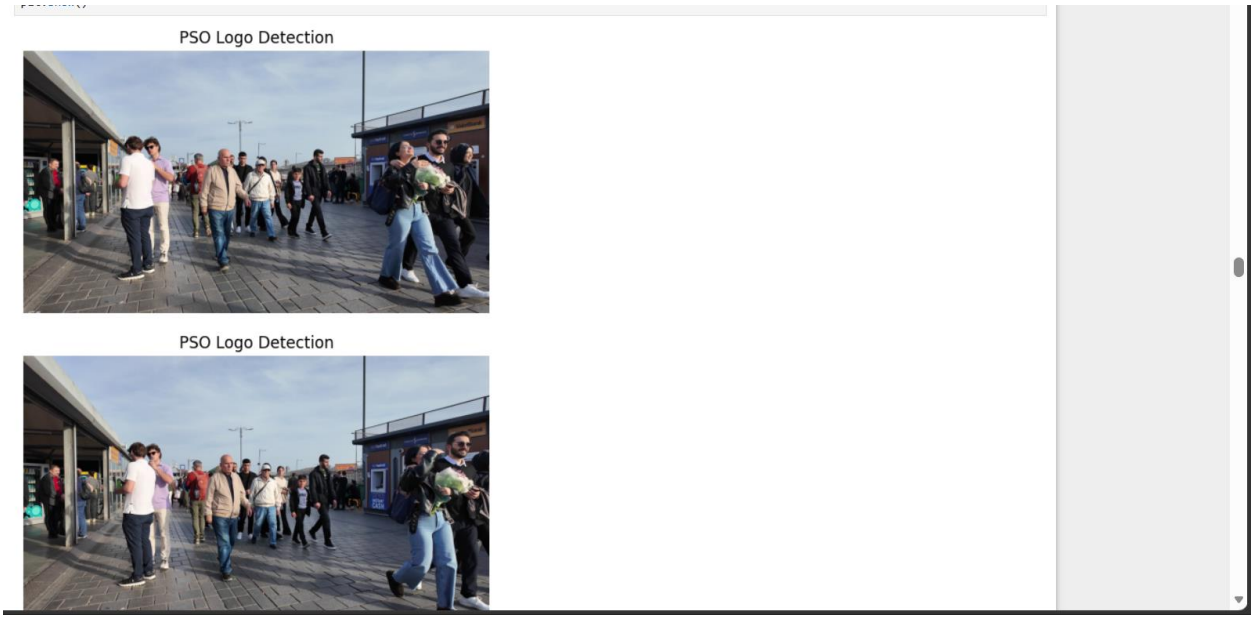
```
Best Match Score: -0.09837893
Animal Not Detected
```

Case Study 5: Logo Detection in Video Frames for Advertisement Monitoring
Title: Optimized Logo Spotting in Video Advertisements Using PSO
Scenario: Media analytics companies need to verify whether brand logos appear during sponsored video content.
Implementation:
□ Given a company logo (template), PSO searches video frames for occurrences.
□ The fitness function rewards shape and color similarity.
□ Helps quantify brand exposure time.

Real-world application: Used in broadcast analysis, sports sponsorship evaluation, and digital marketing validation.

```
import cv2
import numpy as np
import random
import matplotlib.pyplot as plt
video=cv2.VideoCapture("D:/23108111/Lecture H.mp4")
template=cv2.imread("D:/23108111/HH4.jpg")
if template is None:
    raise Exception("Logo template not found")
template=cv2.resize(template,(0,0),fx=0.5,fy=0.5)
th,tw,_=template.shape
template_gray=cv2.cvtColor(template,cv2.COLOR_BGR2GRAY)
num_particles=5
iterations=5
w_inertia=0.5
c1=1
c2=2
frame_count=0
plt.ion()
while True:
    ret,frame=video.read()
    if not ret:
        break
    frame_count+=1
    if frame_count%5!=0:
        continue
    frame=cv2.resize(frame,(0,0),fx=0.5,fy=0.5)
    h,w,_=frame.shape
    if th>=h or tw>=w:
        continue
    frame_gray=cv2.cvtColor(frame,cv2.COLOR_BGR2GRAY)
    def fitness(x,y):
        x=int(x)
        y=int(y)
        if x<0 or y<0 or x+tw>w or y+th>h:
            return -1
        roi=frame_gray[y:y+th,x:x+tw]
        if roi.shape!=template_gray.shape:
            return -1
```

```
        return cv2.matchTemplate(roi,template_gray,cv2.TM_CCOEFF_NORMED)[0][0]
    positions=[]
    for _ in range(num_particles):
        positions.append(
            np.array([random.randint(0,w-tw),random.randint(0,h-th)],dtype=float))
    velocities=[np.zeros(2) for _ in range(num_particles)]
    pBests=positions.copy()
    pBest_scores=[fitness(p[0],p[1]) for p in pBests]
    gBest=pBests[np.argmax(pBest_scores)]
    gBest_score=max(pBest_scores)
    for _ in range(iterations):
        for i in range(num_particles):
            r1=random.random()
            r2=random.random()
            velocities[i]=(w_inertia*velocities[i]+c1*r1*(pBests[i]-positions[i])+c2*r2*(gBest-positions[i]))
            positions[i]=positions[i]+velocities[i]
            positions[i][0]=np.clip(positions[i][0],0, w-tw)
            positions[i][1]=np.clip(positions[i][1],0,h-th)
            score=fitness(positions[i][0],positions[i][1])
            if score>pBest_scores[i]:
                pBests[i]=positions[i].copy()
                pBest_scores[i]=score
            if score>gBest_score:
                gBest=positions[i].copy()
                gBest_score=score
    x,y=int(gBest[0]),int(gBest[1])
    if gBest_score>0.7:
        cv2.rectangle(frame,(x,y),(x+tw,y+th),(0,255,0),3)
        cv2.putText(frame,"Logo Detected",(x,y-10),cv2.FONT_HERSHEY_SIMPLEX,0.7,(0,255,0),2)
    frame_rgb=cv2.cvtColor(frame,cv2.COLOR_BGR2RGB)
    plt.clf()
    plt.imshow(frame_rgb)
    plt.title("PSO Logo Detection")
    plt.axis("off")
    plt.pause(0.001)
video.release()
plt.ioff()
plt.show()
```



https://github.com/Habiba-Bibi/